ANDROID

1 - ¿En qué momento del ciclo de vida de una Activity se ejecuta el método `onCreate()`?

El método onCreate() es el primer método en ejecutarse en el ciclo de vida de una Activity, apenas se crea la activity se ejecuta el método. En el caso de la primer activity de la aplicación, se ejecuta onCreate() en el instante en el que el usuario apretó el icono de la aplicación.

2 - ¿Qué se debe realizar en el método `onCreate()`?

Como se ha explicado anteriormente, el método onCreate() representa el estado previo a la creación de la creación de la activity. En el estado actual, la activity no es visible y no es posible interactuar con ella. En este método se debe inicializar el layout de la activity a mostrar, e inicializar variables de la vista, en memoria.

3 - ¿De qué superclase heredan las Activities? ¿Qué métodos son necesarios implementar mínimamente en un activity propia?

Toda activity hereda de la clase Activity. Mínimamente requiere del método onCreate() en donde se define el Layout a usar, y se inicializan las variables de la vista.

4 - Describa los siguientes elementos del Framework de sensores de Android.

Las clases principales para trabajar con los sensores son las siguientes:

- **`Class SensorManager`**: Permite obtener los distintos sensores que posee el smartphone / dispositivo y también permite añadir nuevos SensorEventListeners para cada Sensor. Se usa para acceder a los distintos sensores y crear instancias de los mismos.
- **`Class Sensor`**: Representa a un sensor en sí. Es la clase genérica donde heredan los distintos tipos de sensores.
- **`Class SensorEvent`**: Los eventos que envían los sensores. Estos eventos contienen los valores que envía el sensor dentro de un array. En otras palabras, representa el evento de un sensor y tiene información sobre este. Es una clase donde representa toda la información correspondiente al sensor y al evento relacionado.
- **`Interface SensorEventListener`**: Toda clase que es capaz de “escuchar” eventos de sensores. Con estos podemos obtener los valores de los sensores. Recibe notificaciones del SensorManager cuando hay datos nuevos. Se registra objeto sobre él y el sensor mantiene informado si su valor cambia.

5 - ¿Para qué y cómo se utilizaría el método `onAccuracyChanged()`?

El método `onAccuracyChanged()` en Android se utiliza en el contexto de los sensores y se invoca cuando la precisión de un sensor cambia. Este método es parte de la interfaz `SensorEventListener` y se emplea principalmente para monitorear y responder a variaciones en la precisión de los datos que proporciona un sensor específico.

Parámetros:

- `sensor`: es una instancia del objeto Sensor que indica cuál sensor ha cambiado de precisión.
- `accuracy`: es un valor entero que indica el nuevo nivel de precisión del sensor.

6 - Mencione de qué forma se podrían utilizar al menos dos sensores en Android para poder interactuar con el entorno del Usuario.

Ejemplo 1: Entonces, con el SensorManager podemos obtener dos sensores, por ejemplo, el acelerómetro y el giroscopio. Con el sensor manager agregamos dos listeners, uno para el primer sensor y otro para el segundo. Con estos listeners vamos a obtener los valores de cada sensor.

El acelerómetro indica la aceleración del celular. Mientras que el giroscopio tiene como valor el ángulo de giro.

Ejemplo 2: Podrían usarse 2 sensores para una aplicación que desbloquea automáticamente un celular apoyado en una superficie con la pantalla hacia abajo. Al separarlo de la superficie provoca un cambio en el sensor de proximidad (o el lumínico en su defecto) y esto sumado a el ángulo de giro captado por el giroscopio, podría desbloquear la pantalla.

Ejemplo 3

- Sensor lumínico: Al detectar una baja en la luz podría aumentar el brillo de la pantalla.
- Giroscopio: Se puede captar el ángulo de giro y dependiendo de este se desbloquea la pantalla.

7 - ¿Con qué clase del Framework de Android se realiza la suscripción a los reportes de los sensores?

Es la clase **`SensorManager`**. Este listener será una clase que implemente la interfaz **`SensorEventListener`.**

8 - ¿Para qué utilizaría el componente ContentResolver? ¿Y el ContentProvider?

El componente **`ContentResolver`** es útil para comunicar nuestra aplicación con otra enviándole datos. **`ContentProvider`** se encarga de proveer la información necesaria a un **`ContentResolver`** para que interactúe con algún componente del Sistema. En otras palabras, un **`ContentResolver`** permite acceder a la información de un **`ContentProvider`**. Sólo están activos cuando responden a ellos.

- Con este componente puedo reaccionar ante eventos como “compartir”.
- Para enviar estadísticas de reproducción desde mi aplicación de reproducción de música hacia otra aplicación que junte estadísticas.
- Una aplicación que requiere acceder a la base de datos de los contactos de Android.
- **ContentResolver** lo utilizará para acceder a los datos provistos por otras aplicaciones mediante un **ContentProvider**.
- **ContentProvider** se utiliza para el intercambio de datos entre aplicaciones.

Respuesta ChatGPT

El componente `ContentResolver` en Android se utiliza para interactuar con un `ContentProvider`, que es una interfaz para compartir datos entre aplicaciones de manera segura. A través de un ContentResolver, una aplicación puede acceder, modificar o eliminar datos gestionados por `ContentProviders`, siempre que tenga los permisos necesarios.

Por ejemplo, si una aplicación necesita acceder a la base de datos de contactos de Android, puede usar el `ContentResolver` para realizar consultas, obtener resultados, o incluso actualizar datos de contactos. Los `ContentProviders` se emplean en Android para estructurar el acceso a datos compartidos, como contactos, archivos multimedia y configuraciones, y el `ContentResolver` actúa como intermediario para gestionar esas operaciones de manera eficiente y segura.

9 - ¿Qué objeto se utiliza para iniciar una Activity desde otra Activity?

Se utiliza Intents (donde estará toda la información): Se crea un Intent desde la activity llamadora, los parámetros se almacenan en el Intent, se inicia la nueva activity y dentro de esta se recuperan los parámetros con un objeto

bundle (recupera la información en la nueva Activity). Para realizar esto, se crea un Intent con los siguientes parámetros: clase de activity origen y clase de activity destino. Luego se inicia la nueva activity con el Intent.

```
Intent intent = new Intent(this, MainActivity.class); startActivity(intent);
```

10 - En Android hay dos tipos de Intents. Explique la diferencia entre cada uno de ellos.

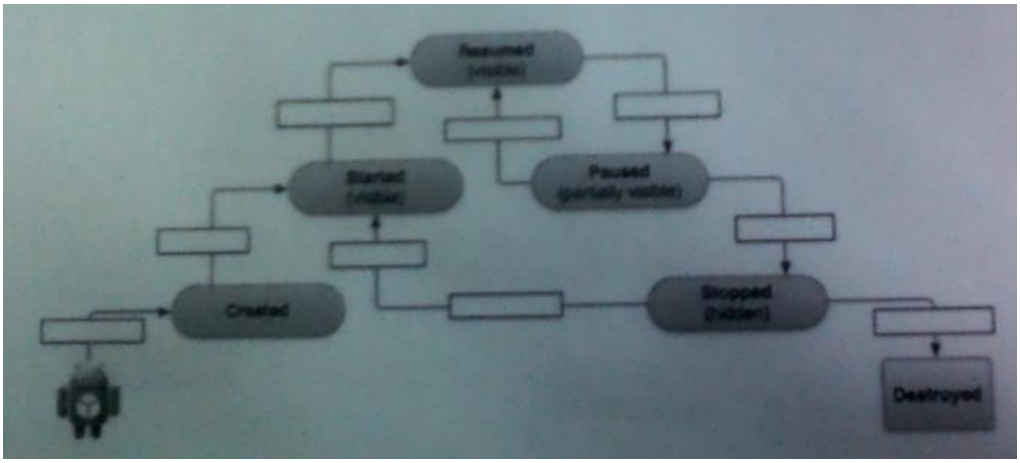
Los Intents son mensajes entre activities, este objeto permite la navegación y el envío de mensajes entre componentes.

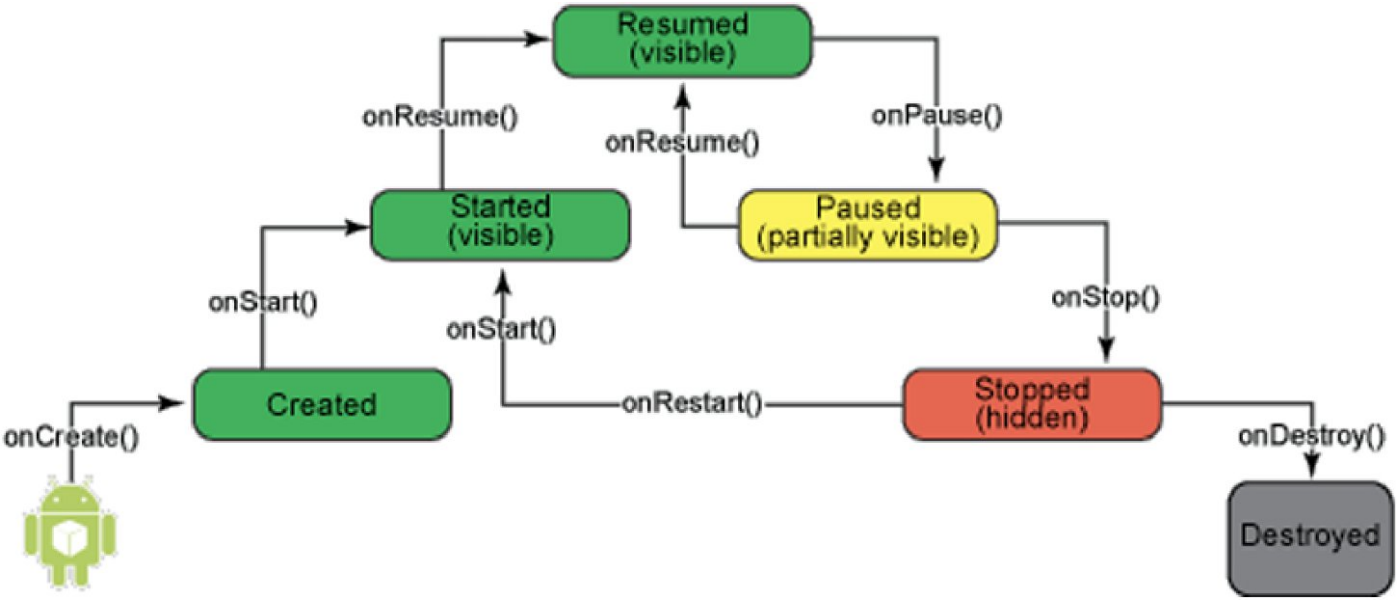
- **Intent Explicito:** Ejecutamos un componente en específico mediante su nombre (el nombre de clase completamente calificado).
- **Intent Implícito:** En el implícito se entrega una referencia genérica sobre alguna condición, y aquellas entidades que cumplan ese requisito serán presentadas como candidatas para la tarea. Ejemplo: Compartir un sitio web en una red social y nos muestra cual. No se nombra al componente explícitamente, se declara una acción general para realizar, lo que permite que un componente de otra aplicación la maneje.

11 - Describa cuál es la relación existente entre el sensor de tipo *TYPE_LINEAR_ACCELERATION* y el *TYPE_ACCELEROMETER*.

El primero mide la aceleración descontando la gravedad y el segundo por cambios de gravedad y movimiento. La relación que encuentro en común es que **ambos miden aceleración**.

12 - Complete el nombre de los métodos invocados para acceder a cada estado del ciclo de vida de actividades. Ejemplifique comentando paso a paso la ejecución de una actividad hipotética que inicie, sea ocluida parcialmente por un diálogo, y luego resuma su ejecución hasta luego ser finalizada normalmente por el usuario al ejecutar una acción específica sobre la misma.





13 - Enumere los principales componentes del Framework de Android.

- **Activities**: Son las pantallas con las cuales interactúa el usuario. El contenido de ella se representa con Views que pueden ser TextViews, Buttons, etcétera. En las activities encontramos todo lo que va a estar visible. Están implementadas sobre la clase Activity.
- **Service**: Se trata de clases que realizan procesamiento en segundo plano, sin interfaz de usuario. Por ejemplo, con ellas se pueden realizar descargas por red, o reproducir música.
- **BroadcastReceiver / Broadcast Annonuncement**: Se trata de un componente capaz de recibir y reaccionar ante Broadcasts de Android. Estos Broadcast son mensajes o eventos del sistema, o definidos por el usuario. Con ellos se puede conocer el estado de la batería cada vez que cambia, o reaccionar ante una notificación.
- **ContentProvider** y **ContentResolver**: Son componentes que permiten realizar comunicación entre distintas aplicaciones. Los ContentResolvers permiten acceder a datos provistos por ContentProviders.
- **Intent**: Es un componente que permite enviar datos a través de activities. A su vez, es el iniciador de activities, services y broadcasts. Por ejemplo, me permite realizar una lectura de código QR y obtener la información almacenada en el código, o pasar de una activity hacia otra.

14 - Explica los mecanismos de backgound vistos

Tenemos 3 mecanismos, Service, Assync Task y Threads.

- **Service**: Hay dos tipos, Service e IntentService. Estos no poseen interfaz.
    - El primero extiende de la clase `Service` por lo que se crea sobre el mismo hilo que la activity, es decir en el main thread. Esto puede generar bloqueantes, para evitarlo, dentro del servicio se crea un hilo secundario y se comunicara con a activity mediante técnicas como broadcast o handler.
    - Por otro lado, **Intent Service**, extiende de la clase `IntentService` y esta clase automáticamente crea un hilo secundario, para comunicarse con la activity emplea lo mismo, broadcast o hanlder, entre otros.
- **Assync Task**: Permite generar hilos secundarios, en segundo plano que pueden comunicarse fácilmente con el hilo principal. Por ejemplo, se puede realizar operaciones en segundo plano y muestran los resultados en el hilo principal. Se basa en 4 funciones:
    - `onPreExecute()`: Ejecuta sobre el hilo principal
    - `doInBackground()`: Hace todo lo que tenga que hacer el hilo en segundo plano
    - `onProgressUpdate()`: Es quien muestra los datos por pantalla a medida que ejecuta el hilo secundario
    - `onPostExecute()`: Es el que finaliza toda la ejecución.
- **Threads**: Cuando se crea la aplicación, se crea un hilo principal que es el que manejara todo lo que ocurra. A la vez se pueden crear hilos secundarios, que se comunicaran con el principal mediante un handler. Estos corren concurrentemente al hilo principal.

15 - ¿Qué componente del Framework de Android se utiliza para ejecución en Background?

Antes de responder a la pregunta, se puede hacer una mención honorífica a la clase **Thread** de Java, que permite ejecutar hilos en segundo plano. Estos no son parte del framework de Android, sino que son parte de la biblioteca estándar de Java.

Dentro del framework de Android contamos con los siguientes componentes background:

- **Services**: Ya explicados en la pregunta anterior. También podemos encontrar otro tipo de Service llamado IntentService, que, a su vez, ejecuta en un hilo separado al hilo de la UI.
- **AsyncTask**: Un componente que permite realizar operaciones en un hilo en segundo plano mostrar resultados en el hilo principal. Se lo suele utilizar para tareas de corta duración. Es un wrapper alrededor de Thread.

16 - Explique la diferencia entre un Service y un IntentService.

Ambas clases permiten realizar una tarea background sin interfaz de usuario, sin embargo, su mayor diferencia radica en el hilo en el cual este procesamiento se ejecuta. En el caso de Service, a menos que se indique lo contrario, se ejecutará el procesamiento en el hilo principal. Mientras que el IntentService ejecutará el procesamiento en un hilo secundario.

17 - ¿Para qué utilizaría un Servicio en Android?

Utilizaría un servicio para las descargas de archivos o reproducción de música, ya que son cosas que transcurren en segundo plano y no necesita la intervención / interacción por parte del usuario.

18 - ¿Cuál es el propósito del archivo Android Manifest? ¿Para qué se utiliza?

Situado en la raíz de nuestras aplicaciones como AndroidManifest.xml, es un archivo de configuración donde podemos aplicar las configuraciones básicas de nuestra app. Establece el nombre del package de la aplicación, para identificar de manera única a la aplicación.

En dicho archivo se encuentra la información general de la aplicación, como su nombre, permisos y features que usa. También se encuentran definidos los componentes a utilizar, y los IntentFilters.

Permite definir los componentes de la aplicación, es decir, Activities, Services, Content Providers etc., estableciendo qué clases implementan cada componente. Determina que Activity será lanzada inicialmente. Declara el nivel mínimo de la API que Android requiere para la aplicación. Se declaran los permisos que necesita la aplicación para realizar determinadas tareas, cómo el acceso a contactos del teléfono o realizar llamadas de teléfono, accediendo a zonas restringidas de la API e interactuando con otras aplicaciones.

19 - ¿Qué sensores utilizaría para determinar la posición de un dispositivo en Android?

Geomagnético + Giroscopio + Acelerómetro.

20 - ¿Qué ocurre con las variables miembro del objeto de la clase que representa a la actividad en primer plano si cambia la orientación del dispositivo y se provoca la destrucción completa y posterior recreación de la actividad?

Vuelve a empezar todo el ciclo de vida.

21 - ¿Para qué es utilizado el entorno de desarrollo Android NDK?

NDK: Native Development Kit. El NDK permite realizar **compilación cruzada** desde mi computadora hacia distintos dispositivos Android. A su vez, permite recrear el entorno de Android, para facilitar el desarrollo. Es utilizado para reconstruir el entorno de destino en la máquina donde hacemos la compilación. lo hacemos mediante las herramientas que nos brinda.

22 - ¿Cómo se puede persistir datos en Android? Explicar las distintas maneras

Existen varias maneras de persistir datos en Android. Enumero las dos más comunes:

- **SharedPreferences**: Permite guardar datos utilizando un esquema key-value al estilo db de UNIX o Redis. Esta técnica guarda los registros bajo la metodología “clave-valor”.
- Otra manera es mediante archivos que se almacenan en la memoria interna o externa del dispositivo.
- **SQLite**: Permite utilizar un motor de bases de datos embebido en el framework de desarrollo. Este motor de bases de datos permite realizar consultas en lenguaje SQL sobre tablas representadas en archivos. Cuando se cierra la aplicación, los datos son guardados en un archivo que se almacena dentro del almacenamiento interno y a este archivo solo puede acceder el usuario ya que es privado.

24 - ¿En qué parte del ciclo de vida es cuando se da el onDestroy()?

25 - ¿Ventajas del MVP y diferencia del MVC?

26 - Seleccione un sensor del Framework de Android e indique para qué y cómo lo utilizaría.

27 - ¿Para qué se utilizan los filtros en el Archivo Manifest de Android?

28 - ¿Qué es en Android el ADB?

29 - Describa cómo se implementaría adecuadamente la comunicación entre dos fragmentos instanciados en una actividad (Por ejemplo, un Button “Incrementador” en un Fragment y un Label “contador” en otro).

La comunicación entre Fragments es posible gracias a que hay una ventana o activity de intermediario, es decir que los fragmentos tienen en común la ventana a la que pertenecen.

Lo único que debemos hacer es que la ventana implemente una interfaz y sobrescriba sus métodos.

Después uno de los fragmentos, en este ejemplo el fragmento que envía información, creará una instancia de esta interfaz. Se usará el polimorfismo para decir que la interfaz Comunicador es la ventana principal, esto es gracias a que la ventana principal implementa a la interfaz Comunicador.

30 - Describa el uso y propósito de la clase *FragmentTransaction*. Ejemplifique su uso, ya sea en código o en prosa.

Para realizar transacciones de fragmentos en tu actividad (como agregar, quitar o reemplazar un fragmento), debes usar las API de FragmentTransaction.

```
FragmentManager manager = getSupportFragmentManager();
FragmentTransaction transaction = manager.beginTransaction();
transaction.replace(R.id.output, fragment);
transaction.commit();
```

31 - Suponga que desea visualizar el estado del clima en la interfaz principal de su aplicación del TP2. Para ello, puede hacer uso de una API externa de un Web Service, la cual provee la información necesario, utilizando el protocolo REST a través de una URI. En base a esto, ¿qué mecanismos utilizaría para incorporar dicha funcionalidad en su aplicación? ¿Qué consideraciones debería tener en cuenta para mostrar los datos en la pantalla?

Utilizaría el mismo mecanismo que hemos utilizado en el TP2 para comunicar la aplicación con el Web Service proveído por la cátedra. HttpURLConnection para realizar la comunicación con el Web Service, y Handler para poder realizar la comunicación con la interfaz del usuario y mostrar los resultados por pantalla. Esta comunicación con el Web Service la realizaría en un hilo secundario, utilizando la clase Thread de la biblioteca estándar de Java, dado que me es más cómoda.

32 - Suponga que en una nueva aplicación de Android desea alertar al usuario de que no posee conexión a Internet. ¿Qué mecanismos utilizaría para incorporar dicha funcionalidad? ¿Qué consideraciones debe tener en cuenta para saber que no hay Internet?

Para cumplir con este requerimiento, antes de realizar cualquier consulta HTTP se puede verificar que haya conexión a Internet. Para realizar esto, se puede obtener la información de la red con NetworkManager, verificar que haya información de Red (dado que si es null significa que no se está conectado a ninguna red), y una vez que se haya verificado si se está conectado a una Red, se deberá verificar que esta Red esté conectada a Internet. Para realizar esta verificación rápidamente, se puede realizar un ping a un servidor que siempre suele estar up, como google.com.

En el caso de que no se esté conectado a Internet, se puede comunicar esto al hilo de la UI utilizando un Handler.

33 - Explica las diferencias entre en una comunicación bloqueante y asincrónica con respecto peticiones REST

Con respecto a la comunicación bloqueante o sincrónica, esta es enviar una petición al servidor y realizar una espera bloqueante, es decir detener la ejecución hasta obtener respuesta, inhabilitando las interfaces de las activities. Esto puede ocasionar fallos ya que nos puede dar errores como la aplicación no responde, ya que se detiene la ejecución del hilo principal.

Con respecto a la comunicación asincrónica, esta realiza la petición al servidor, pero el main thread no detiene su funcionamiento por lo que no se bloquea la ejecución. Cuando el servidor tenga respuesta y envié un HTTP Response, como respuesta de un HTTP Request se hará un callback, el mainthread procesara la respuesta.

34 - Enumere los distintos métodos que se podrían utilizar para comunicar una aplicación con las placas Intel Galileo - ¿De qué manera se puede comunicar un programa desarrollado para Android (APK) con las Placas Intel Galileo?

Bluetooh, necesitamos 3 componentes:

- **BluetoohAdapter**: Todos los componentes referidos al Bluetooh. Se utiliza para realizar operaciones con el Bluetooth y permite obtener los datos del Bluetooth del Arduino.
- **BluetoohDevice**: Se encarga del emparejamiento y nos da acceso a la transmisión de datos
- **BluetoohSocket**: Permite la conexión y desconexión, es decir, permite recibir y enviar datos.

Web Server: Consiste en una interfaz que nos da acceso a ciertas funcionalidades que ofrece un servidor. El acceso a estos servicios se hace mediante protocolos web como HTTP.

- Local: Arduino + celular.
- Servidor: Arduino + Celular apuntan a un Servidor.
- Nube: El servidor deja toda la información en la nube y un dispositivo se conecta.

MQTT: está orientado a IOT, consume poco ancho de banda, ofrece seguridad y requiere que la transferencia de datos sea baja. usa topología estrella.

35 - Los fragmentos fueron añadidos a partir de Android 3.0, ¿Qué necesidad disparó principalmente su introducción?

Android introduce los fragmentos en Android 3.0 (nivel de API 11), principalmente para admitir diseños de IU más dinámicos y flexibles en pantallas grandes, como las de las tablets. Como la pantalla de una tablet es mucho más grande que la de un teléfono, hay más espacio para combinar e intercambiar componentes de la IU.

Otra respuesta: La necesidad de poder modificar partes de la vista dinámicamente en base a acciones del usuario sin necesidad de destruir y construir activities para cada evento disparador (lo que llevaría a una estructura de aplicaciones poco clara y robusta). La clase FragmentTransaction permite agregar y quitar estos.

PREGUNTAS COLOQUIO

1 - ¿Cómo implementaste los sensores en tu trabajo practico?

2 - ¿Cómo implementaste la navegación entre activities en tu trabajo practico?

3 - Explicar los mecanismos de sincronización que utilizo en su Trabajo Practico

En nuestra aplicación utilizamos un mecanismo de sincronización llamado Synchronized, el cual nos ayudó a indicar a que a ciertas regiones del código solo podría ingresar un subproceso a la vez. También utilizamos RunOnUiThread el cual nos permitió ejecutar acciones específicas desde un thread secundario. También se podría aclarar que utilizamos HttpUrlConnection para sincronizar las peticiones de nuestra app con la API de la catedra.

4 - ¿Qué métodos de sincronización utilizaron en el TP?

El método de sincronización que utilizamos en el TP es la palabra reservada de Java synchronized. Esto permite encerrar una región crítica, para que sólo un hilo del proceso pesado sea capaz de atravesar dicha región. Es una especie de mutex que se pide antes de comenzar la región crítica y que se libera luego de pasarla.